

Agri-Logistics Optimizer

66050174 ธนบดี กาญจนโอษฐ์

โครงการนี้เป็นส่วนหนึ่งของรายวิชาโครงการเชิงปฏิบัติ (05506239)

หลักสูตรปริญญาวิทยาศาสตรบัณฑิต วิทยาการคอมพิวเตอร์

ภาควิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ปีการศึกษา 2568

1. ที่มาและความสำคัญของโครงการ

ปัญหาหรือโอกาสที่ต้องการแก้

ในปัจจุบัน ธุรกิจจำหน่ายปัจจัยการผลิตทางการเกษตร เช่น ปุ๋ย เมล็ดพันธุ์ และสารเคมีเกษตร ประสบปัญหาสำคัญในด้านการบริหารจัดการขนส่งสินค้า เนื่องจากสินค้าส่วนใหญ่มีน้ำหนักมากและมีปริมาณเยอะ ทำให้ลูกค้าส่วนใหญ่ไม่สามารถขนย้ายกลับด้วยตนเองได้ในครั้งเดียว หลายร้านจึงบริการการจัดส่งปุ๋ยให้กับลูกค้าถึงหน้าสวนหรือแปลงเกษตร แต่การนัดหมายในการขนส่งยังเกิดปัญหาอีกว่าพนักงานขับรถขนส่งที่จะไปส่งของไม่รู้ถึงถึงที่ลูกค้าแจ้ง เวลาที่จะไปส่งจึงต้องขอพื้นที่กว้างๆหรือแลนด์มาร์คสำคัญในการที่จะไปถึงแล้วจึงโทรศัพท์ถามลูกค้าต่อการที่จะไปถึงจุดหมาย

เหตุผลที่เลือกทำหัวข้อนี้

ลูกค้าส่วนใหญ่ที่มาซื้อสินค้าปัจจัยการผลิตทางการเกษตรในปัจจุบัน พบว่าลูกค้าส่วนใหญ่นิยมเดินทางมาเลือกซื้อสินค้าที่หน้าร้านด้วยตนเอง เนื่องจากต้องการเปรียบเทียบราคา ตรวจสอบคุณภาพสินค้า และปรึกษาการใช้งานกับผู้ขายโดยตรง อย่างไรก็ตาม ด้วยลักษณะของสินค้าเกษตร (เช่น ปุ๋ย หรือสารปรับปรุงดิน) ที่มีน้ำหนักมากและมีปริมาณการสั่งซื้อต่อครั้งค่อนข้างสูง ลูกค้าจึงมักใช้วิธีการสั่งซื้อหน้าร้านและให้ทางร้านดำเนินการจัดส่งให้ไปยังแปลงเกษตรหรือสวน

กลุ่มผู้ใช้และบริบทของงาน

ผู้ประกอบการ/เจ้าของร้านใช้ระบบ POS ในการบันทึกยอดขาย จัดการสต็อกสินค้า และบริหารจัดการคิวการขนส่ง พนักงานขับรถขนส่งใช้ระบบเพื่อดูรายการงานที่ต้องส่งและใช้แผนที่นำทางไปยังพิกัดที่ลูกค้าปักหมุดไว้ได้อย่างแม่นยำ ลูกค้าผู้มาซื้อสินค้าหน้าร้านที่ต้องการบริการจัดส่งโดยสามารถระบุตำแหน่งแปลงเกษตรหรือจุดวางสินค้าผ่านแผนที่ในระบบได้อย่างชัดเจน

คุณค่าหรือประโยชน์ที่คาดว่าจะได้รับ

เมื่อระบบถูกนำไปใช้งาน คาดว่าจะช่วยสร้างคุณค่าที่สำคัญ คือเพิ่มความแม่นยำในการจัดส่งสินค้าถึงเป้าหมายด้วยพิกัด GPS ลดความผิดพลาดจากการหลงทาง ซึ่งส่งผลโดยตรงต่อการลดต้นทุนด้านเวลาและค่าเชื้อเพลิงของผู้ประกอบการ ในขณะเดียวกันยังช่วยยกระดับมาตรฐานการบริการ ทำให้ลูกค้าเกิดความพึงพอใจจากการได้รับสินค้าที่ตรงเวลา และไม่ต้องเสียเวลาในการบอกเส้นทางซ้ำซ้อนในการซื้อครั้งถัดไปเนื่องจากระบบมีการจัดเก็บประวัติพิกัดไว้อย่างเป็นระบบ

ขอบเขตเบื้องต้นและสมมติฐานสำคัญ

โครงการนี้ครอบคลุมการพัฒนา Web Application ที่รองรับการทำงานตั้งแต่หน้าการขาย (POS) การจัดการฐานข้อมูลพิกัดลูกค้า การจัดการคิวขนส่ง ไปจนถึงส่วนแสดงผลสำหรับคนขับ โดยมีสมมติฐานสำคัญคือ ข้อมูลแผนที่จากผู้ให้บริการ API มีความรายละเอียดเพียงพอสำหรับพื้นที่เกษตรกรรม และพนักงานขับรถสามารถใช้ระบบนำทางได้อย่างต่อเนื่อง ซึ่งหากสมมติฐานเหล่านี้

เป็นไปตามคาด ระบบจะสามารถลดขั้นตอนการทำงานที่ซ้ำซ้อนและเพิ่มประสิทธิภาพการขนส่งสินค้าเกษตรได้อย่างมีนัยสำคัญ

2. งานและเทคโนโลยีที่เกี่ยวข้อง

ในการพัฒนาเว็บแอปพลิเคชันเพื่อบริหารจัดการหน้าร้านและการขนส่งสินค้าเกษตร มีเทคโนโลยีและแนวคิดที่เกี่ยวข้อง ดังนี้:

2.1 PostGIS (Spatial Database Extension) เป็นส่วนขยายของฐานข้อมูล PostgreSQL ที่ทำให้สามารถจัดการข้อมูลเชิงพื้นที่ (Spatial Data) ได้อย่างมีประสิทธิภาพ ในโครงการนี้ใช้ PostGIS เพื่อจัดเก็บพิกัดละติจูดและลองจิจูดของลูกค้าในรูปแบบ geography และใช้โอเปอเรเตอร์ <-> (Nearest Neighbor) ในการค้นหาจุดส่งที่ใกล้ที่สุด

2.2 First-Fit Algorithm (Bin Packing Problem) เป็น Heuristic Algorithm ที่ใช้แก้ปัญหาการจัดวางสิ่งของลงในกล่องที่มีขนาดจำกัด ในโครงการนี้ใช้เพื่อจัดกลุ่มออเดอร์สินค้าเกษตรที่มีน้ำหนักมากให้ลงตัวกับความจุสูงสุดของรถแต่ละคัน

2.3 Next.js และ TypeScript (Modern Web Development) Next.js เป็น React Framework ที่ใช้สำหรับการพัฒนา Frontend และ TypeScript เป็นภาษาทางโปรแกรมที่มีการระบุชนิดของตัวแปร (Type Safety)

2.4 Mapbox API เป็นบริการ API สำหรับแสดงผลแผนที่และจัดการข้อมูลพิกัดเชิงตำแหน่ง ใช้สำหรับฟังก์ชันการปักหมุดตำแหน่งแปลงเลขตรของลูกค้านำทางสำหรับคนขับรถ

ในการส่งสินค้าทางการเกษตรที่เน้นให้ลูกค้าปักหมุดส่งสินค้าลงบนแปลงเกษตรนั้น ต้องการแผนที่ที่มีความแม่นยำและใช้งานได้ง่าย รวมไปถึงการปรับแต่งในการออกแบบการใช้งานของแผนที่ที่ทำได้ง่าย โดยใช้ Mapbox ที่รองรับการใช้ร่วมกับ Express.js ในการทำ Backend และใช้ Frontend Next.js ที่รองรับการทำ Responsive Design ผ่าน TailwindCSS ได้ดีซึ่งจำเป็นสำหรับพนักงานขับรถที่ต้องใช้งานผ่านอุปกรณ์เคลื่อนที่ ส่วน TypeScript ช่วยลดข้อผิดพลาดในการส่งข้อมูลและพัฒนา ในส่วนของ Database ใช้ Postgres ที่มี Extensions เสริม คือ PostGIS ที่รองรับการเก็บข้อมูลที่เป็นพิกัดบนพื้นผิวโลก ทำให้การเก็บหรือการจัดการกับข้อมูลที่ลูกค้าปักหมุดนั้นสะดวกขึ้นในการพัฒนา

3. ขอบเขต เป้าหมาย และข้อกำหนด

3.1 เป้าหมายของโครงการ คือการทำระบบ POS(Point of Sale) ที่ช่วยในการจัดการกับการซื้อสินค้าหน้าร้านและอำนวยความสะดวกในการปึกหมุดเพื่อส่งสินค้าให้กับลูกค้า และการเพิ่มประสิทธิภาพในการจัดกลุ่มสินค้าลงรถบรรทุกเพื่อลดต้นทุนด้านโลจิสติกส์

3.2 ขอบเขตที่ทำ

Functional Requirement

- 1.ระบบต้องสามารถเพิ่มรายการสินค้าเข้าสู่ออเดอร์ในการสั่งซื้อได้
- 2.ระบบต้องการล็อกอินแยกผู้ใช้งานระหว่างคนขับขนส่งสินค้า พนักงานหน้าร้าน เจ้าของร้านหรือแอดมิน
- 3.ระบบต้องสามารถให้ลูกค้าปึกหมุดในการขนส่งสินค้าลงบนแผนที่ในระบบได้
- 4.ระบบต้องสามารถจัดรายการน้ำหนักสินค้าที่พอดีกับน้ำหนักในการขนส่งของรถได้
- 5.ระบบต้องสามารถแสดงเส้นในการขนส่งของแต่ละออเดอร์ให้กับคนขับรถขนส่งของได้

Non-functional Requirements

- 1.Security: ข้อมูลรหัสผ่านผู้ใช้งานต้องถูกเข้ารหัสด้วย bcrypt และสื่อสารผ่านโปรโตคอลที่มีความปลอดภัยโดยใช้ JWT
- 2.Usability: หน้าจอฝั่งคนขับต้องรองรับการแสดงผลแบบ Responsive เพื่อใช้งานบนอุปกรณ์เคลื่อนที่ได้สะดวก

3.3 ขอบเขตที่ไม่ทำหรือข้อจำกัดของโครงการ

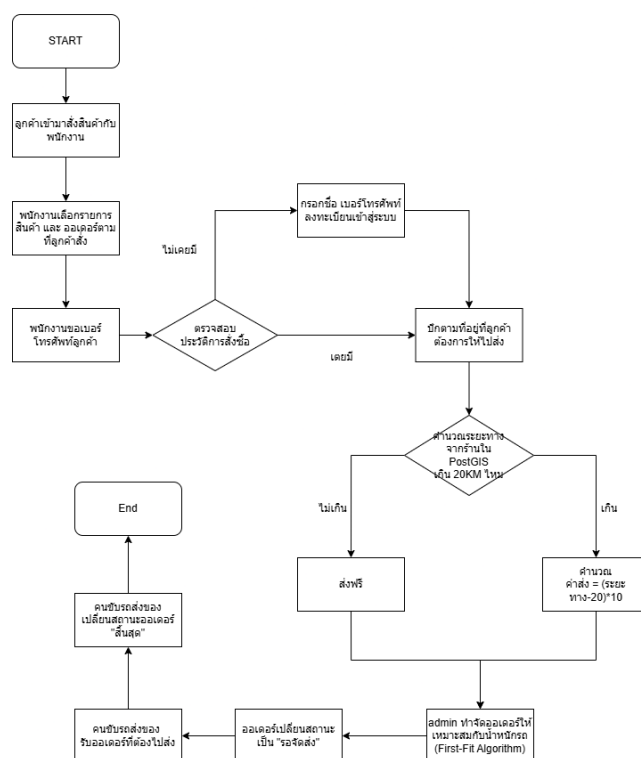
- 1.ระบบถูกพัฒนาและทดสอบบนสภาพแวดล้อม Localhost เป็นหลัก
- 2.ระบบไม่มีระบบสมัครสมาชิก (Register) สำหรับผู้ทั่วไป โดยจะใช้ข้อมูลผู้ใช้ที่ถูกสร้างในฐานข้อมูลเพื่อทดสอบสิทธิ์การใช้งาน
- 3.ระบบไม่รองรับการค้นหาพิกัดผ่านช่องค้นหาที่อยู่ ผู้ใช้ต้องปึกหมุดตำแหน่งที่ต้องการลงบนแผนที่โดยตรงเท่านั้น
- 4.ไม่มีระบบเพิ่มหรือจัดการสต็อกสินค้าในฝั่งแอดมิน โดยจะใช้ข้อมูลสินค้าคงคลังคงที่มีอยู่ในฐานข้อมูลเท่านั้น
- 5.ระบบต้องเชื่อมต่ออินเทอร์เน็ตตลอดเวลาเพื่อดึงข้อมูลแผนที่และภาพถ่ายดาวเทียมจาก Mapbox API

4. การออกแบบหรือแนวทางเชิงเทคนิค

4.1 แผนภาพลำดับการทำงาน (System Flowchart)

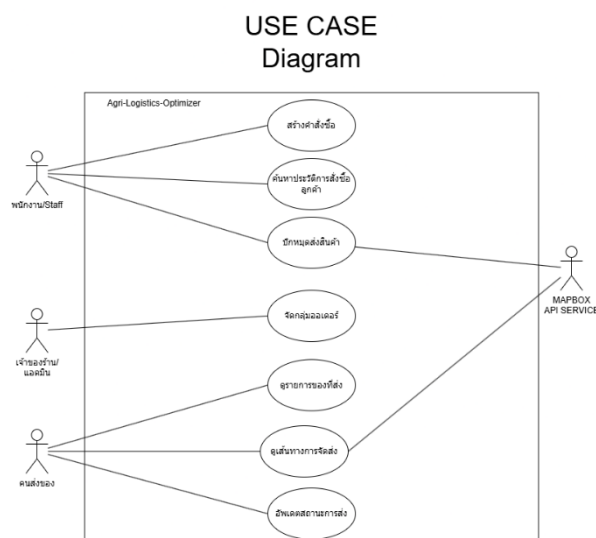
แสดงขั้นตอนการทำงานของระบบ (Business Logic) ตั้งแต่เริ่มต้นจนถึงสิ้นสุดกระบวนการ

Agri-Logistics Optimizer



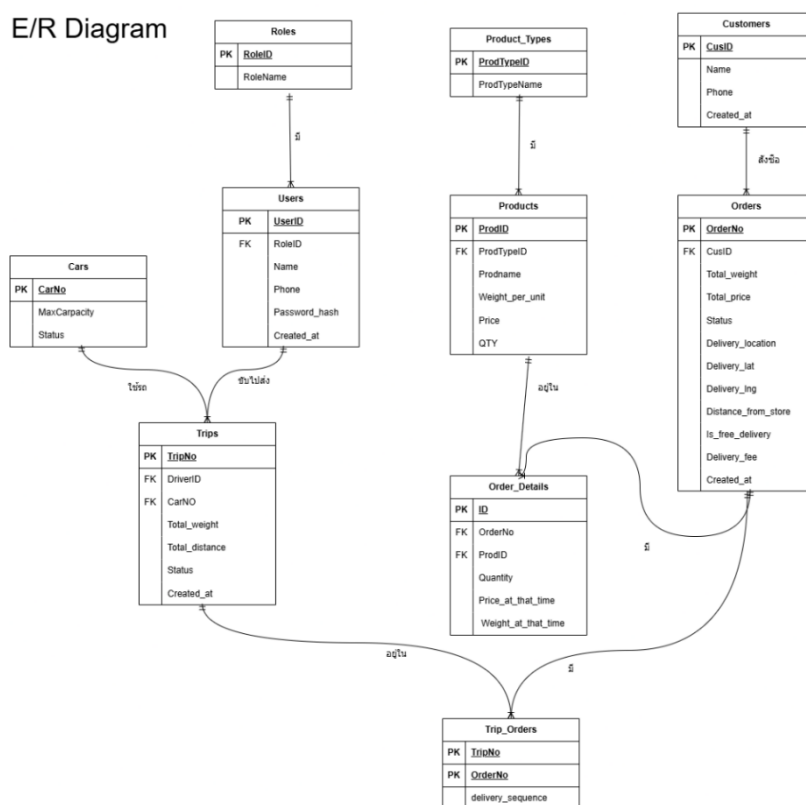
4.2 แผนภาพกรณีใช้งาน (Use Case Diagram)

แผนภาพนี้แสดงความสัมพันธ์ระหว่างผู้ใช้งานและฟังก์ชันการทำงานหลักโดยแบ่งตามสิทธิ์การใช้งานของพนักงานหน้าร้าน เจ้าของร้าน/แอดมิน คนขับรถ



แผนภาพความสัมพันธ์ของข้อมูล (ER Diagram)

ตารางหลัก (Entities) ประกอบด้วย Users, Roles, Cars, Orders, Customers, และ Products ส่วนรายละเอียด (Weak Entities) Order_Details: เก็บรายการสินค้าในแต่ละออเดอร์ รวมถึงราคาน้ำหนัก ณ เวลาที่สั่งซื้อ Trip_Orders: ทำหน้าที่เชื่อมโยงออเดอร์เข้ากับรอบการเดินรถ (Trips) พร้อมเก็บลำดับการจัดส่ง (delivery_sequence) ที่ได้จากการคำนวณ ความสัมพันธ์: ใช้ Roles ในการกำหนดสิทธิ์เข้าถึง



5. เครื่องมือ เทคโนโลยี และทรัพยากรที่ใช้

Frontend : Typescript,TailwindCSS,NextJS

Backend : Javascript,NodeJS,ExpressJs

Database : Postgres+PostGIS

External API : Mapbox Directions API

Authentication : JWT,bcrypt

Testing Tools : Postman

สภาพแวดล้อมที่ใช้พัฒนาและทดสอบ

Asus TUF Gaming F15 CPU: Intel Core i5-10300H ,GPU : NVIDIA GTX 1650, RAM 16GB และ SSD 512GB

6. การทดสอบหรือการประเมินผล

การทดสอบตามกรณีการใช้งาน (Use Case Testing): เป็นการทดสอบแบบ Black-Box Testing เพื่อยืนยันว่าระบบทำงานได้ตามเงื่อนไขทางธุรกิจ (Business Logic) ที่กำหนดไว้

Test Case ID	Test Case Scenario	Test Case	Pre-conditions	Test Steps	Test Data	Expected Results	Actual Results	Execution Result
TC-01	การจัดการคำสั่งซื้อและพิกัด	ตรวจสอบการคำนวณค่าส่งตามระยะทางจริง	พนักงานล็อกอินเข้าสู่ระบบและมีพิกัดร้านค้าตั้งต้น	1. เลือกสินค้าเข้าออเดอร์ 2. ปักหมุดพิกัดบนแผนที่ห่างจากร้าน 25 กม. 3. ตรวจสอบค่าขนส่งที่แสดง	พิกัด Lat/Lng ที่ ระยะทาง 25 กม. จากร้าน	ระบบต้องแสดงค่าส่ง 50 บาท ตามสูตร (25-20)*10	PASS	PASS
TC-02	การจัดกลุ่มออเดอร์	ตรวจสอบการจัดกลุ่มออเดอร์โดย First-Fit	มีออเดอร์สถานะ pending และมีรถว่างในระบบ	1. สร้างออเดอร์ 3 รายการ 2. กดปุ่ม 'Optimize Trips' ในหน้าแอดมิน 3. ตรวจสอบการแบ่งทริป	น้ำหนักออเดอร์: 5000, 1200, 800 กก. ความจุ: 6000 กก.	ออเดอร์ 5000 และ 800 กก. ต้องอยู่ทริปเดียวกันเพื่อแยกทริปใหม่	PASS	PASS
TC-03	การจัดลำดับเส้นทางขนส่ง	ตรวจสอบการจัดลำดับการส่งด้วย PostGIS Operator <->	ทริปถูกสร้างขึ้นและมีออเดอร์มากกว่า 2 รายการ	1. แอดมินกดจัดลำดับเส้นทางในทริป 2. ตรวจสอบค่า delivery_sequence ในฐานข้อมูล	ข้อมูลทริปที่มีจุดส่งกระจายตัวหลายตำแหน่ง	ออเดอร์ที่อยู่ใกล้พิกัดร้านที่สุดต้องมีลำดับเป็น 1 และเรียงลำดับถัดไปตามระยะทาง	PASS	PASS
TC-04	การส่งมอบสินค้าและสถานะ	ตรวจสอบการอัปเดตสถานะออเดอร์เมื่อคนขับส่งของสำเร็จ	คนขับได้รับมอบหมายทริปและเข้าสู่หน้ารายการงาน	1. คนขับกดยืนยันการส่งสินค้าในแอป 2. ตรวจสอบสถานะออเดอร์ในหน้าแอดมิน	Order Status: pending -> delivered	สถานะออเดอร์ในฐานข้อมูลต้องเปลี่ยนเป็น delivered และบันทึกเวลาสำเร็จทันที	PASS	PASS

การทดสอบระดับส่วนหลังบ้าน (Backend API Testing) ด้วย Postman

ผู้พัฒนาได้เลือกใช้เครื่องมือ Postman ในการทำ Integration Testing เพื่อจำลองการส่งคำขอ (Request) ไปยัง API Endpoint ต่างๆ ของระบบ

GET /api/products response: 200 OK

POST /api/auth/login response: 200 OK

POST /api/orders/check-delivery-fee response: 200 OK

POST /api/orders/create response: 201 Created

PATCH /api/orders/ORD-00001/delivered response: 200 OK

POST /api/optimize/run response: 200 OK

7. ข้อจำกัด ประเด็นจริยธรรม และแนวทางพัฒนาต่อ

7.1. ข้อจำกัดของโครงการ

- 1.ระบบถูกพัฒนาและทดสอบบน Localhost เป็นหลัก
- 2.ไม่มีระบบสมาชิกสำหรับบุคคลทั่วไป โดยใช้ข้อมูลผู้ใช้ Staff, Admin, Driver ที่เตรียมไว้ในฐานข้อมูลเท่านั้น
- 3.ระบบไม่รองรับการค้นหาด้วยที่อยู่ (Geocoding) ผู้ใช้ต้องทำการปิกหมุดตำแหน่งด้วยตนเองบนแผนที่เท่านั้น
- 4.ไม่มีระบบจัดการสต็อกสินค้าแบบ Dynamic โดยใช้ข้อมูลสินค้าคงคลังคงที่จากฐานข้อมูล
- 5.ระบบจำเป็นต้องเชื่อมต่ออินเทอร์เน็ตตลอดเวลาเพื่อดึงข้อมูลแผนที่และภาพถ่ายดาวเทียมจาก Mapbox API

7.2. ประเด็นด้านจริยธรรมและความเป็นส่วนตัว

- 1.ข้อมูลพิกัดที่ตั้งสวนหรือแปลงเกษตรของลูกค้าเป็นข้อมูลที่ระบุตัวตนถึงเคสสถานได้ ระบบจึงจำกัดสิทธิ์การเข้าถึงให้เห็นเฉพาะผู้ที่มีหน้าที่จัดส่งและผู้จัดการระบบเท่านั้น.
- 2.มีการเข้ารหัสผ่านด้วย bcrypt และใช้ JWT ในการยืนยันตัวตนเพื่อป้องกันการสวมสิทธิ์เข้าถึงข้อมูลคำสั่งซื้อ.
- 3.สูตรการคำนวณค่าขนส่งที่ชัดเจน $\text{Dist} - 20 \times 10$ ช่วยให้ลูกค้ามั่นใจในความยุติธรรมของราคาที่ระบบคำนวณให้.

7.3. แนวทางพัฒนาต่อ

- 1.พัฒนาระบบจัดการสต็อกสินค้า
- 2.พัฒนาแอปพลิเคชันสำหรับคนขับรถเพื่อรองรับการแจ้งเตือนและการนำทางและรวมไปถึงการติดตามที่อยู่ของรถแบบ Realtime
- 3.ย้ายระบบจาก Localhost ไปยังระบบ Cloud เพื่อให้สามารถใช้งานผ่านอินเทอร์เน็ตได้ทุกที่ทุกเวลา.
- 4.เพิ่มระบบ Geocoding เพื่อให้ผู้ใช้สามารถค้นหาที่ตั้งได้ด้วยชื่อหมู่บ้านหรือที่อยู่บ้านเลขที่ แทนการปิกหมุดด้วยมือเพียงอย่างเดียว

8. เอกสารอ้างอิง

PostGIS Project Steering Committee. (2026). *PostGIS 3.4.2 Manual*. [Online].

<https://postgis.net/docs/manual-3.4/>.

Mapbox. (2026). *Mapbox GL JS API Reference*. [Online].

<https://docs.mapbox.com/mapbox-gl-js/api/>.

OpenJS Foundation. (2026). *Express 4.x API Reference*. [Online].

<https://expressjs.com/en/4x/api.html>.

Microsoft. (2026). *TypeScript Documentation*. [Online].

<https://www.typescriptlang.org/docs/>.

Tailwind Labs Inc. (2026). *Tailwind CSS Documentation*. [Online].

<https://tailwindcss.com/docs>.

การใช้ AI เข้ามาช่วยเหลือในการทำงาน

Gemini 3.1 Fast

“ในการขนส่งสินค้าเกษตรเช่นปุยที่หนักหลายกิโลแล้วต้องมีการจัดลงรถ ควรใช้ Bestfit หรือ Firstfit มากกว่า”

"ออกแบบตารางการทดสอบ (Test Case) สำหรับระบบโลจิสติกส์ โดยครอบคลุมหัวข้อ Test Case ID, Scenario, Steps, Expected Results และ Actual Results"

Gemini 3.1 Pro

“ขอ FrontEnd Login โดยกรอกแค่ เบอร์โทรและรหัสผ่าน ใช้ NextJS ”

Cluade sonnet 4.6

“ถ้าใช้ในการวาง route ในการส่งหลายจุด map ไหนดีบ้างขอฟรี”

“อยากทดลองใช้ Typescript + Mapbox ขอโค้ดและวิธี setup เบื้องต้น ขอแค่ทดลองเชื่อมดูเฉยๆขึ้นหน้าเว็บแบบง่ายๆ”

“ช่วยสร้าง FrontEnd โปรเจคของฉันหน่อย Flow ประมาณว่า ลูกค้ามาสั่งแล้วถ้ามาสั่งอีกรอบเราก็ค้นพิกัดจากเบอร์โทรลูกมาได้เลยไม่ต้องปึกใหม่อะไรจ้ะ จริงๆระบบหลักๆคือ ลูกค้าเข้ามาเลือกสินค้า และจะเก็บข้อมูลลูกค้าโดยเป็นชื่อ เบอร์โทร ที่อยู่ตามที่ลูกค้าเลือกปึกหมดเวลาลูกค้าซื้อของ และเก็บ role id ด้วย เลือกสินค้าเสร็จส่งของกับพนักงาน พนักงานสร้าง order ที่ระบบเลือกสินค้า จากนั้นให้ลูกค้าแจ้งพิกัดหรือกดปึกหมดเมื่อปึกหมดก็ดึงพิกัดลง database แล้วคำนวณว่าอยู่ในระยะรัศมี 20 กิโลเมตร หรือไม่ในการส่งฟรี ถ้าหากเกินจะมีค่าส่งเพิ่มเติม เรียบร้อยแล้วก็สร้างออเดอร์เมื่อสร้างออเดอร์เสร็จก็รอให้แอดมินมาจัดออเดอร์ลงรถแล้วคราวนี้ก็ดูออเดอร์ทั้งหมดที่ต้องส่งตามสถานะรายการ โดยจากนั้นออเดอร์จะจัดลงรถตามน้ำหนักที่เหมาะสมของน้ำหนักสินค้า แล้วคนขับก็มีหน้าที่ดูสินค้าขับตามหมดไป แล้วอัปเดตสถานะของออเดอร์ด้วย โดยผู้ที่ล็อกอินนี้จะมีแค่ Staff Admin Driver โดยใช้ NextJS Typescript tailwind”

“โดยที่หน้าแรกเลือกสินค้า แล้วกดเพิ่มสินค้าเสร็จไปหน้าถัดไปเพื่อให้แมพใหญ่ขึ้นง่ายต่อการหาการกดและการจัดการข้อมูลการขนส่ง”

“โปรเจคนี้อวดสอบและวัดประสิทธิภาพอย่างไรได้บ้าง แนะนำวิธีมาหน่อย”

“order มันควรจะแสดง order แล้วจัดน้ำหนักลงรถตามน้ำหนักรถ ส่วนDriver-tasks ก็ควรแค่ Driver Login เข้ามาดูว่ามีออเดอร์อะไรต้องส่งบ้างแล้ว แมพก็โชว์รูปแบบโตๆ”

“Mockup user Driver Staff Admin ลง Database หน่อย เอาเบอร์จำง่ายๆ”